

JAVA TECHNICAL INTERVIEW PREPARATION

Complete Beginner → Advanced → Tricky Interview Guide

Questions in this edition: 101

Major topics: Java basics, OOP, Strings, exceptions, concurrency, collections, generics, Java 8+, JVM, GC, JDBC, and modern Java.

Preparation Roadmap

Stage	Focus
1. Foundation	Syntax, data types, variables, operators, control flow
2. Core Java	Strings, methods, constructors, keywords, access control
3. OOP	Encapsulation, inheritance, polymorphism, abstraction, interfaces
4. Professional Java	Exceptions, concurrency, collections, generics
5. Modern Java	Lambdas, streams, Optional, records, sealed classes
6. Internals	JVM, memory, JIT, garbage collection
7. Backend Essentials	File I/O, Date/Time, JDBC
8. Interview Mode	Tricky outputs, design scenarios, comparisons

Table of Contents

1. Java Basics — 10 questions
2. General Java — 7 questions
3. Strings — 6 questions
4. Methods, Constructors & Keywords — 5 questions
5. OOP — 11 questions
6. Access Control — 2 questions
7. Exception Handling — 5 questions
8. Multithreading & Concurrency — 7 questions
9. Wrapper Classes — 3 questions
10. Collections — 6 questions
11. Generics — 3 questions
12. Java 8+ Features — 6 questions
13. JVM & Memory — 4 questions
14. Garbage Collection — 2 questions
15. Object Class — 2 questions
16. Enums — 1 questions
17. Annotations — 1 questions
18. File I/O — 1 questions
19. Date & Time — 1 questions
20. JDBC — 2 questions
21. Modern Java — 3 questions
22. Tricky Interview Questions — 8 questions
23. Scenario-Based Questions — 5 questions

1. Java Basics

Q1. What are primitive data types in Java?

Answer

1. Java has eight primitive types: byte, short, int, long, float, double, char, and boolean.
2. They store simple values directly rather than object references; their sizes are defined by the Java language except boolean's representation is not specified as a fixed byte size.
3. Numeric widening is generally automatic, while narrowing requires an explicit cast and can lose information.
4. Local variables do not receive default values; instance and static fields do receive type-specific default values.

Java Code Example

```
int age = 22;
double salary = 45000.50;
char grade = 'A';
boolean active = true;
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Primitive variables are like labeled boxes holding the actual simple value.

Interview Tip: Do not claim boolean is guaranteed to occupy exactly 1 byte in Java.

Q2. What is the difference between widening and narrowing conversion?

Answer

1. Widening converts a smaller numeric type to a wider compatible type and normally happens implicitly.
2. Narrowing converts a wider type to a smaller type and requires an explicit cast.
3. Widening usually preserves the numeric value, while narrowing may overflow or lose fractional information.
4. Casting between unrelated reference types follows different rules and may fail at runtime.

Java Code Example

```
int x = 10;
double y = x; // widening

double p = 10.8;
int q = (int) p; // narrowing
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Moving water from a small container into a larger one is usually safe; pouring a large quantity into a smaller container can lose content.

Interview Tip: Mention both compile-time conversion rules and possible data loss.

Q3. What is the difference between local, instance, and static variables?

Answer

1. A local variable is declared inside a method, constructor, or block and exists only within that scope.
2. An instance variable belongs to an object, so each object normally has its own copy.
3. A static variable belongs to the class and is shared by instances of that class.
4. Local variables must be initialized before use, while fields receive default values.

Java Code Example

```
class Counter {
    int instanceCount;
```

```
static int totalCount;

void test() {
    int localCount = 1;
}
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A local variable is like a person's temporary note; an instance field is that person's property; a static field is a shared notice board.

Interview Tip: Be precise about scope versus lifetime; they are related but not identical.

Q4. What is the difference between == and ==?

Answer

1. = is the assignment operator and stores a value in a variable.
2. == is the equality operator and compares primitive values or reference identity for objects.
3. For objects, == does not compare object contents; equals() is normally used for logical equality.
4. Using == where equals() is required is one of the most common Java interview mistakes.

Java Code Example

```
int a = 10;
int b = 10;
System.out.println(a == b); // true
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Assignment is putting something into a box; equality is asking whether two things match according to the applicable comparison rule.

Interview Tip: Always distinguish primitive comparison from reference comparison.

Q5. What is the difference between break, continue, and return?

Answer

1. break terminates the nearest applicable loop or switch.
2. continue skips the remainder of the current loop iteration and proceeds with the next iteration.
3. return exits the current method and may optionally provide a value.
4. They differ mainly in how far control flow moves: iteration, loop/switch, or method.

Java Code Example

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    if (i == 5) break;
    System.out.println(i);
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

continue means 'skip this turn'; break means 'stop this loop'; return means 'leave this method'.

Interview Tip: For nested loops, state exactly which loop a break affects.

Q6. What is an infinite loop?

Answer

1. An infinite loop is a loop whose termination condition never becomes false.
2. It can be intentional for servers or event loops, but accidental infinite loops are bugs.
3. A while(true) loop can be stopped using break, return, interruption, or external process termination depending on the design.
4. Interviewers often test whether you understand the update expression and loop condition.

Java Code Example

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++; // prevents an infinite loop
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A traffic light stuck on green can keep traffic moving forever; an infinite loop similarly has no successful exit condition.

Interview Tip: Check the condition, state changes, and reachable exit path.

Q7. What is switch in Java?

Answer

1. switch selects one execution path based on a selector expression.
2. Modern Java supports switch with integral types, enum, String, and newer switch expressions depending on the Java version.
3. break is traditionally used to prevent fall-through in statement-style switch cases.
4. Switch expressions can return values and use arrow labels, making intent clearer in modern Java.

Java Code Example

```
String role = "ADMIN";
switch (role) {
    case "ADMIN" -> System.out.println("Full access");
    case "USER" -> System.out.println("Limited access");
    default -> System.out.println("Unknown");
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A switch is like routing a request to one of several known service counters.

Interview Tip: Know the difference between traditional switch statements and modern switch expressions.

Q8. What is type casting?

Answer

1. Type casting explicitly tells Java to treat a value as another compatible type.
2. Primitive casts can convert numeric values; narrowing may lose information.
3. Reference downcasting requires the actual object to be compatible with the target type.
4. Invalid reference casts can throw ClassCastException at runtime.

Java Code Example

```
Object obj = "Java";
String s = (String) obj;
System.out.println(s);
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Casting is like saying, 'I know this object is actually of this more specific type; treat it accordingly.'

Interview Tip: Distinguish compile-time type from runtime object type.

Q9. What is operator precedence?

Answer

1. Operator precedence determines which operators are evaluated first when an expression has multiple operators.
2. Parentheses can be used to make the intended evaluation order explicit.
3. Multiplication, division, and remainder generally bind more tightly than addition and subtraction.
4. When unsure, use parentheses rather than relying on readers to remember precedence.

Java Code Example

```
int result = 2 + 3 * 4;  
System.out.println(result); // 14
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

It is similar to mathematical order of operations.

Interview Tip: Be ready to evaluate expressions involving ++, &&, ||, and the ternary operator.

Q10. What is a ternary operator?

Answer

1. The ternary operator is a compact conditional expression using condition ? value1 : value2.
2. It produces a value, unlike a traditional if statement which is a statement.
3. It is useful for simple two-way selections but becomes unreadable when nested heavily.
4. Both selected expressions should be type-compatible enough for Java to determine a result type.

Java Code Example

```
int age = 20;  
String result = age >= 18 ? "Adult" : "Minor";  
System.out.println(result);
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

It is a compact 'choose A or B' decision.

Interview Tip: Do not replace complex business logic with deeply nested ternaries.

2. General Java

Q11. What is Java?

Answer

1. Java is a general-purpose, class-based programming language designed for portability, reliability, and large-scale application development.
2. Java source is compiled into bytecode, which is executed by a JVM implementation on a target platform.
3. Its standard library and ecosystem support web services, enterprise applications, Android history, tools, data processing, and many backend systems.
4. Java combines object-oriented programming with features such as automatic memory management, exceptions, concurrency APIs, and generics.

Java Code Example

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello, Java");  
    }  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Java aims to let developers write code against a stable platform abstraction rather than directly targeting one operating system.

Interview Tip: Say 'platform independent through bytecode and a platform-specific JVM,' not simply 'Java is independent of the OS.'

Q12. Why is Java platform independent?

Answer

1. The Java compiler normally converts source code into platform-neutral bytecode.
2. A JVM implementation for each operating system translates or executes that bytecode on the local platform.
3. Therefore the same compiled class file can run on different platforms when compatible Java runtime support exists.
4. The JVM itself is platform dependent; the bytecode is the portable artifact.

Java Code Example

```
javac Hello.java  
# produces Hello.class  
java Hello
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Bytecode is like a common instruction language, while each JVM acts as the local interpreter/engine.

Interview Tip: The precise phrase is 'Write Once, Run Anywhere' with a JVM on the target platform.

Q13. What are JDK, JRE, and JVM?

Answer

1. JVM is the runtime engine that executes Java bytecode and manages runtime concerns such as memory and garbage collection.
2. JRE traditionally means the JVM plus runtime libraries needed to run applications; modern JDK distributions are the normal way to obtain the runtime.
3. JDK is the developer kit containing tools such as javac plus the runtime components.

4. For interviews, explain the conceptual relationship while acknowledging that modern Java distributions do not necessarily ship a separate JRE product.

Java Code Example

```
javac Hello.java
java Hello
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Think of JVM as the engine, runtime libraries as the operating environment, and JDK as the developer toolbox plus runtime.

Interview Tip: Avoid stating that every modern JDK installation contains a separately installable JRE directory.

Q14. What is bytecode?

Answer

1. Bytecode is the instruction format stored in .class files after Java source compilation.
2. The JVM loads and verifies bytecode before execution.
3. Bytecode enables the same compiled class to be used by JVM implementations on different platforms.
4. The JVM can interpret bytecode and/or compile frequently executed code to native machine code using JIT techniques.

Java Code Example

```
public class Demo {
    public static void main(String[] args) {
        System.out.println("Bytecode");
    }
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

It is an intermediate representation between Java source and native machine execution.

Interview Tip: Do not call bytecode machine code; it is JVM-oriented intermediate code.

Q15. What happens when a Java program runs?

Answer

1. The source is compiled by javac into bytecode.
2. The JVM starts, loads required classes through class loaders, verifies/links them, and initializes classes when required.
3. Execution begins through the main entry point when launching a conventional application.
4. The JVM interprets bytecode and may JIT-compile hot code, while managing runtime memory and garbage collection.

Java Code Example

```
javac Demo.java
java Demo
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A compiler prepares the program; the JVM then loads, prepares, executes, and optimizes it.

Interview Tip: Separate compile-time errors from class-loading, linkage, and runtime errors.

Q16. Why was Java created?

Answer

1. Java originated at Sun Microsystems under James Gosling and the Green project in the early 1990s.

2. The language was designed for portability and reliability across different devices and environments.
3. Its C/C++-influenced syntax was combined with automatic memory management and a virtual-machine execution model.
4. The language became strongly associated with networked and enterprise software as its ecosystem expanded.

Java Code Example

```
System.out.println("Portable software");
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

The original goal was not merely to create another C-like language; portability and controlled execution were major design motivations.

Interview Tip: Remember James Gosling and the original name Oak.

Q17. Why is main declared public static void?

Answer

1. public allows the launcher/runtime to access the entry method from outside the class.
2. static lets the method be invoked without first creating an object of the class.
3. void means the entry method does not return a value to the caller.
4. String[] args receives command-line arguments passed to the application.

Java Code Example

```
public static void main(String[] args) {  
    for (String arg : args) System.out.println(arg);  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

It is a class-level entry point that the launcher can invoke and that receives text arguments.

Interview Tip: Know each keyword separately; do not memorize the whole signature as one phrase.

3. Strings

Q18. Why is String immutable in Java?

Answer

1. A String's character content cannot be changed after the String object is created.
2. Immutability makes String safe to share across code and supports string pooling without one user changing another user's value.
3. It also makes cached hash codes and security-sensitive uses more reliable because the content does not mutate.
4. Operations that appear to modify a String actually create or return another String when a different value is needed.

Java Code Example

```
String s = "Java";  
s = s.concat(" Interview");  
System.out.println(s);
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A printed word on a sealed label cannot be edited; you create a new labeled value instead.

Interview Tip: String variables can change references even though String objects are immutable.

Q19. What is the String pool?

Answer

1. The String pool is a JVM-managed pool of canonical String instances, especially used for string literals.
2. Equal literals can reuse the same pooled object, which makes reference comparison sometimes appear to work.
3. `new String("Java")` explicitly creates a distinct String object even when the same text is already pooled.
4. Interning can canonicalize strings, but unnecessary interning can increase memory pressure.

Java Code Example

```
String a = "Java";  
String b = "Java";  
String c = new String("Java");  
System.out.println(a == b);  
System.out.println(a == c);  
Expected Output:  
true  
false
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A shared dictionary can keep one copy of a common word, while new String asks for a separate object.

Interview Tip: Never use `==` as the general-purpose content comparison for Strings.

Q20. String vs StringBuilder vs StringBuffer?

Answer

1. String is immutable, so repeated concatenation can create many intermediate objects.
2. StringBuilder is mutable and is generally preferred for repeated single-threaded string construction.
3. StringBuffer is mutable and synchronizes its commonly mutating methods, which can add overhead.
4. Choose based on mutability and concurrency requirements rather than assuming StringBuffer is always safer.

Java Code Example

```
StringBuilder sb = new StringBuilder();  
sb.append("Java");
```

```
sb.append(" Interview");
System.out.println(sb);
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

String is a sealed document; StringBuilder is an editable draft; StringBuffer is an editable draft with synchronized operations.

Interview Tip: For normal local string building, StringBuilder is usually the default choice.

Q21. What is the difference between == and equals() for Strings?

Answer

1. == compares whether two references point to the same object.
2. String.equals() compares the character content of the strings.
3. String literals may share pooled objects, so == can return true for identical literals.
4. Using equals() expresses the intended content comparison and works correctly for independently created equal strings.

Java Code Example

```
String a = new String("Java");
String b = new String("Java");
System.out.println(a == b);
System.out.println(a.equals(b));
Expected Output:
false
true
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Two books can contain identical text while still being two different physical books.

Interview Tip: Mention null-safety when comparing unknown references, e.g. "Java".equals(value).

Q22. What does compareTo() do?

Answer

1. compareTo() performs lexicographic comparison and returns a negative value, zero, or a positive value.
2. Zero means the compared strings are equal according to their ordering comparison.
3. It is useful for sorting and ordering rather than simply testing equality.
4. String comparison is case-sensitive unless a different comparison strategy is chosen.

Java Code Example

```
System.out.println("apple".compareTo("banana"));
System.out.println("cat".compareTo("cat"));
Expected Output:
a negative value
0
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

It is similar to comparing words in a dictionary.

Interview Tip: Do not promise a particular negative or positive number; the contract guarantees sign, not magnitude.

Q23. What is String concatenation?

Answer

1. Concatenation combines string values into a new string.
2. The + operator can concatenate strings and automatically converts non-string operands using string conversion rules.
3. For a small number of operations, + is readable; in loops, a StringBuilder is often more efficient.
4. Compile-time constant string concatenations can be folded by the compiler.

Java Code Example

```
String name = "Java";  
String text = name + " Interview";  
System.out.println(text);
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

It joins pieces of text into one resulting value.

Interview Tip: Explain why repeated concatenation inside a large loop can have avoidable allocation cost.

4. Methods, Constructors & Keywords

Q24. What is method overloading?

Answer

1. Overloading means defining multiple methods with the same name but different parameter lists in the same class.
2. The difference must involve parameter types, number, or order; changing only the return type is not enough.
3. The compiler selects the applicable overload based primarily on the compile-time argument types.
4. Overloading is compile-time polymorphism.

Java Code Example

```
class MathUtil {  
    int add(int a, int b) { return a + b; }  
    double add(double a, double b) { return a + b; }  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A restaurant can have the same service name but different menus depending on the order you place.

Interview Tip: Return type alone cannot overload a method.

Q25. What is method overriding?

Answer

1. Overriding occurs when a subclass provides a compatible implementation of an inherited instance method.
2. The selected implementation is based on the runtime object's class when dynamic dispatch applies.
3. The overriding method cannot reduce access and has restrictions on checked exceptions and return types.
4. Static methods are hidden rather than overridden, and private methods are not overridden.

Java Code Example

```
class Animal { void sound() { System.out.println("Animal"); } }  
class Dog extends Animal { @Override void sound() { System.out.println("Dog"); } }
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A child class replaces inherited behavior for an instance method while preserving the method contract.

Interview Tip: Always mention runtime dispatch when explaining overriding.

Q26. Is Java pass-by-reference or pass-by-value?

Answer

1. Java is strictly pass-by-value.
2. For primitives, the copied value is the primitive value itself.
3. For objects, the copied value is the reference value, so both variables can refer to the same object.
4. Reassigning the parameter does not change the caller's reference, but mutating the referenced object can be visible to the caller.

Java Code Example

```
class Box { int value; }  
static void change(Box b) { b.value = 20; b = new Box(); b.value = 30; }
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A copy of the reference is passed, not the caller's variable itself.

Interview Tip: Saying 'objects are passed by reference' is technically incorrect in Java.

Q27. What is a constructor?

Answer

1. A constructor initializes a newly created object and has the same name as its class.
2. A constructor has no return type, not even void.
3. If a class declares no constructor, the compiler may provide a default no-argument constructor.
4. Constructors can be overloaded, but they are not inherited and cannot be overridden.

Java Code Example

```
class User {  
    String name;  
    User(String name) { this.name = name; }  
}  
User u = new User("Anshu");
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A constructor is like the setup procedure performed when a new object is created.

Interview Tip: Distinguish a compiler-provided default constructor from a user-defined no-argument constructor.

Q28. What does final mean?

Answer

1. A final variable can be assigned only once after definite initialization.
2. A final method cannot be overridden by subclasses.
3. A final class cannot be subclassed.
4. final does not make an object immutable automatically; a final reference can still point to a mutable object whose state changes.

Java Code Example

```
final int x = 10;  
// x = 20; // compile-time error
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

final is a one-time binding restriction, not a universal 'read-only object' keyword.

Interview Tip: Be ready to explain final reference versus immutable object.

5. OOP

Q29. What is a class and what is an object?

Answer

1. A class is a type definition that describes state and behavior.
2. An object is a runtime instance of a class with its own identity and state.
3. The new operator commonly allocates an object and invokes an appropriate constructor.
4. A reference variable stores a reference to an object; it is not the object itself.

Java Code Example

```
class Car { String model; void drive() { System.out.println(model); } }  
Car c = new Car();  
c.model = "Sedan";  
c.drive();
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A class is a blueprint; an object is a built house from that blueprint.

Interview Tip: Do not say a reference variable itself is the object.

Q30. What is encapsulation?

Answer

1. Encapsulation bundles state and behavior inside a class and controls how internal state is accessed.
2. Private fields combined with controlled methods can enforce validation and invariants.
3. It reduces accidental coupling because callers depend on an API rather than internal representation.
4. Encapsulation is about controlling access and protecting state; it is not simply 'using getters and setters'.

Java Code Example

```
class BankAccount {  
    private double balance;  
    public void deposit(double amount) {  
        if (amount > 0) balance += amount;  
    }  
    public double getBalance() { return balance; }  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A bank does not let customers directly edit the ledger; it provides controlled deposit and withdrawal operations.

Interview Tip: Explain why unrestricted setters can weaken encapsulation.

Q31. What is inheritance?

Answer

1. Inheritance lets a subclass reuse and specialize accessible behavior and state from a superclass.
2. Java uses extends for class inheritance and supports single class inheritance.
3. Inheritance models an IS-A relationship when the subtype genuinely satisfies the parent abstraction.
4. Overusing inheritance can create tight coupling; composition is often preferable when behavior is better assembled from components.

Java Code Example

```
class Vehicle { void move() { System.out.println("Moving"); } }  
class Car extends Vehicle { void honk() { System.out.println("Beep"); } }
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A Car is a Vehicle, so the child can reuse the general vehicle contract while adding specialized behavior.

Interview Tip: Do not use inheritance merely for code reuse; model the semantic relationship first.

Q32. What is polymorphism?

Answer

1. Polymorphism means one interface or parent type can represent objects with different concrete behaviors.
2. Compile-time polymorphism is commonly represented by method overloading.
3. Runtime polymorphism occurs with overriding and dynamic method dispatch.
4. A parent reference can refer to a child object, but only members visible through the reference type can be directly accessed at compile time.

Java Code Example

```
class Animal { void sound() { System.out.println("Animal"); } }  
class Dog extends Animal { void sound() { System.out.println("Dog"); } }  
Animal a = new Dog();  
a.sound();  
Expected Output:  
Dog
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A universal remote can control different devices through a common control interface while each device responds differently.

Interview Tip: Separate compile-time overload resolution from runtime override dispatch.

Q33. What is abstraction?

Answer

1. Abstraction exposes essential operations while hiding implementation details.
2. An abstract class can contain abstract methods as well as concrete methods and state.
3. An interface defines a contract and can also contain default, static, and private methods in modern Java.
4. Abstraction reduces the amount of implementation detail that client code must depend on.

Java Code Example

```
abstract class Payment {  
    abstract void pay();  
}  
class CardPayment extends Payment {  
    void pay() { System.out.println("Card payment"); }  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A payment terminal exposes 'pay' while hiding the internal authorization protocol.

Interview Tip: Abstraction is about what is exposed versus how it is implemented.

Q34. Interface vs abstract class?

Answer

1. An interface primarily defines a contract that a class can implement, and a class can implement multiple interfaces.
2. An abstract class can share state, constructors, and common implementation along an inheritance hierarchy.

3. Interfaces can contain abstract, default, static, and private methods under modern Java rules.
4. Choose an interface for capabilities/contracts and an abstract class when shared base state or implementation is genuinely part of the model.

Java Code Example

```
interface Flyable { void fly(); }
class Bird implements Flyable {
    public void fly() { System.out.println("Flying"); }
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

An interface is like a capability contract; an abstract class is more like a partially built common foundation.

Interview Tip: Do not claim interfaces cannot contain implementation in modern Java.

Q35. What is IS-A vs HAS-A?

Answer

1. IS-A represents inheritance or substitutability, such as Dog IS-A Animal.
2. HAS-A represents composition or aggregation, such as Car HAS-A Engine.
3. Composition often reduces coupling because components can be replaced without changing the inheritance hierarchy.
4. Use inheritance when substitutability is valid; use composition when an object owns or collaborates with another component.

Java Code Example

```
class Engine { }
class Car { private Engine engine = new Engine(); }
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A dog belongs to the animal category; a car contains an engine.

Interview Tip: 'Code reuse' alone is not sufficient justification for inheritance.

Q36. What is composition?

Answer

1. Composition builds a class from collaborating objects rather than inheriting their implementation.
2. It supports reuse while keeping the composed components behind a smaller public abstraction.
3. Composition often makes behavior easier to replace, test, and configure.
4. Strong composition commonly means the contained component's lifecycle is controlled by the containing object.

Java Code Example

```
class Engine { void start() { System.out.println("Engine started"); } }
class Car {
    private final Engine engine = new Engine();
    void start() { engine.start(); }
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A car delegates engine work to its engine component instead of becoming a specialized Engine.

Interview Tip: Prefer composition over inheritance when the relationship is behavioral collaboration rather than true substitutability.

Q37. What are equals() and hashCode()?

Answer

1. equals() defines logical equality for objects when a class overrides it appropriately.
2. hashCode() returns an integer used heavily by hash-based collections to select buckets efficiently.
3. If two objects are equal according to equals(), they must return the same hash code.
4. If equals() is overridden, hashCode() should normally be overridden consistently, otherwise HashMap and HashSet behavior can break.

Java Code Example

```
class User {
    int id;
    User(int id) { this.id = id; }
    public boolean equals(Object o) { return o instanceof User u && id == u.id; }
    public int hashCode() { return Integer.hashCode(id); }
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A hash code is like a filing category; equals() is the final check that two records really represent the same logical entity.

Interview Tip: Equal objects must have equal hash codes; unequal objects may still collide.

Q38. What is the difference between tight and loose coupling?

Answer

1. Tight coupling means one class depends strongly on concrete details of another class.
2. Loose coupling depends on abstractions or stable contracts, reducing the impact of implementation changes.
3. Interfaces, dependency injection, and composition are common techniques for reducing coupling.
4. Loose coupling improves testing and maintainability but can add abstractions that are unnecessary for trivial code.

Java Code Example

```
interface Payment { void pay(); }
class Checkout {
    private final Payment payment;
    Checkout(Payment payment) { this.payment = payment; }
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A power socket is loosely coupled to appliances through a standard interface.

Interview Tip: Explain coupling as dependency strength, not simply 'number of classes'.

Q39. What are SOLID principles?

Answer

1. Single Responsibility Principle says a class should have one coherent reason to change.
2. Open/Closed Principle favors extending behavior without modifying stable existing code unnecessarily.
3. Liskov Substitution Principle requires subtypes to remain valid substitutes for their base abstractions.
4. Interface Segregation and Dependency Inversion favor focused contracts and dependence on abstractions rather than concrete details.

Java Code Example

```
interface Printer { void print(); }
interface Scanner { void scan(); }
class OfficeMachine implements Printer, Scanner {
    public void print() { }
    public void scan() { }
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A modular machine with separate printing and scanning contracts avoids forcing every device to implement capabilities it does not need.

Interview Tip: Do not memorize SOLID as five slogans; connect each principle to a design consequence.

6. Access Control

Q40. What are Java access modifiers?

Answer

1. public allows access wherever the type/member is accessible through the language's rules.
2. private restricts access to the declaring top-level class context.
3. protected permits same-package access and qualified subclass access under Java's specific protected rules.
4. Package-private, also called default access, is available when no modifier is written and restricts access to the same package.

Java Code Example

```
class Demo {  
    public int a;  
    private int b;  
    protected int c;  
    int d; // package-private  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Think of access levels as doors with increasingly controlled entry permissions.

Interview Tip: protected is more nuanced than 'accessible everywhere to subclasses'; package and receiver rules matter.

Q41. Can an overriding method reduce access?

Answer

1. No, an overriding method cannot have weaker access than the inherited method.
2. For example, a public parent method cannot be overridden as protected or private.
3. The overriding method may use the same or broader accessibility.
4. This preserves substitutability because callers allowed to use the parent contract must remain allowed to use the child implementation.

Java Code Example

```
class Parent { public void show() {} }  
class Child extends Parent {  
    @Override public void show() {}  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A child replacement cannot close a door that the parent contract promised would be open.

Interview Tip: Also remember that private methods are not overridden.

7. Exception Handling

Q42. What is an exception?

Answer

1. An exception is an abnormal condition represented by an object that disrupts normal execution flow.
2. Exceptions are part of the Throwable hierarchy, which also includes Error.
3. Java provides try, catch, finally, throw, and throws to manage exceptional control flow.
4. Good exception handling should recover when possible, propagate when the caller can decide, and preserve useful diagnostic information.

Java Code Example

```
try {
    int x = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

An exception is like an emergency signal that transfers control to code designed to handle the problem.

Interview Tip: Do not catch Exception everywhere just to suppress failures.

Q43. Checked vs unchecked exceptions?

Answer

1. Checked exceptions are checked by the compiler and must be caught or declared when required by the method signature.
2. Unchecked exceptions are RuntimeException subclasses and do not have the same compile-time catch-or-declare requirement.
3. Checked exceptions can model recoverable conditions where callers are expected to make a decision.
4. Unchecked exceptions commonly represent programming errors or invalid state/arguments, though real designs vary.

Java Code Example

```
void read() throws java.io.IOException {
    throw new java.io.IOException("I/O failed");
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A checked exception is like a task that explicitly requires the caller to acknowledge a possible failure.

Interview Tip: RuntimeException is unchecked, but it is still an Exception and Throwable subtype.

Q44. throw vs throws?

Answer

1. throw is a statement/expression used to actually throw a specific Throwable object.
2. throws appears in a method or constructor declaration to declare checked exceptions that may propagate.
3. throw controls runtime flow; throws communicates part of the method's exception contract.
4. A method can use both when it creates and declares propagation of an exception.

Java Code Example

```
void validate(int age) throws Exception {
    if (age < 18) throw new Exception("Too young");
}
```

```
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

throw is sending the alarm; throws is documenting that the method may pass that alarm to its caller.

Interview Tip: Do not say throws itself throws the exception.

Q45. What is try-with-resources?

Answer

1. Try-with-resources automatically closes objects that implement AutoCloseable when control leaves the try block.
2. It reduces resource-leak bugs and is preferred for streams, readers, database resources, and similar closeable objects.
3. Resources are closed in reverse order of declaration.
4. If both the main operation and close operation fail, the close failure can become a suppressed exception.

Java Code Example

```
try (java.io.BufferedReader br = new java.io.BufferedReader(  
    new java.io.FileReader("data.txt"))) {  
    System.out.println(br.readLine());  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

It is like borrowing equipment with an automatic return desk when the work area closes.

Interview Tip: Know AutoCloseable, close order, and suppressed exceptions.

Q46. What happens if finally contains return?

Answer

1. A return in finally can override a return or exception from try/catch.
2. This can silently discard the original result or exception.
3. Although legal, it is considered poor practice because it makes control flow difficult to reason about.
4. Use finally for cleanup rather than changing the method's result.

Java Code Example

```
static int test() {  
    try { return 1; }  
    finally { return 2; }  
}  
Expected Output:  
2
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

The final return instruction wins, even though the try block already requested a return.

Interview Tip: This is a classic output question; mention why it is dangerous.

8. Multithreading & Concurrency

Q47. What is a thread?

Answer

1. A thread is an execution path within a process.
2. Threads in the same process share process-level resources such as heap memory, while each thread has its own stack and execution state.
3. Threads can improve responsiveness and throughput when work can safely run concurrently.
4. Shared mutable state introduces synchronization and visibility problems that must be designed explicitly.

Java Code Example

```
class Task implements Runnable {  
    public void run() { System.out.println("Task running"); }  
}  
new Thread(new Task()).start();
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A process is like a workshop; threads are workers operating within the same workshop resources.

Interview Tip: Shared heap does not mean every operation is automatically thread-safe.

Q48. start() vs run()?

Answer

1. start() requests the JVM to begin a new thread of execution and eventually invoke run().
2. Calling run() directly is just a normal method call on the current thread.
3. A Thread object can only be started once; attempting to start it again throws IllegalStateException.
4. Thread scheduling is controlled by the runtime/OS and is not guaranteed to follow source order.

Java Code Example

```
Thread t = new Thread(() -> System.out.println(Thread.currentThread().getName()));  
t.start();
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

start opens a new lane; run simply executes the code in the current lane.

Interview Tip: If asked for the most important distinction, say start creates concurrent execution; run does not.

Q49. sleep() vs wait()?

Answer

1. sleep() pauses the current thread for a duration and does not release monitors it already owns.
2. wait() is an Object method used for monitor-based coordination and releases the associated monitor while waiting.
3. wait() must be called while owning the relevant object's monitor, typically inside synchronized code.
4. sleep() is time-based pausing; wait() is generally condition-based coordination with notify/notifyAll.

Java Code Example

```
synchronized (lock) {  
    lock.wait();  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

sleep is taking a timed break while keeping your room key; wait is waiting outside a shared room and releasing the room lock.

Interview Tip: The monitor-release difference is the key interview point.

Q50. What is a race condition?

Answer

1. A race condition occurs when correctness depends on unpredictable timing between concurrent operations.
2. It commonly appears when multiple threads access shared mutable state without adequate atomicity or synchronization.
3. synchronized blocks, locks, atomic classes, immutable design, and concurrent collections can address different forms of the problem.
4. Making one variable volatile does not automatically make compound operations such as count++ atomic.

Java Code Example

```
class Counter {  
    int count;  
    void increment() { count++; }  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Two checkout counters updating the same stock quantity simultaneously can overwrite each other's updates.

Interview Tip: count++ involves read, modify, and write; it is not a single atomic operation.

Q51. synchronized vs volatile?

Answer

1. synchronized provides mutual exclusion and establishes visibility guarantees around monitor locking.
2. volatile primarily provides visibility and ordering guarantees for a variable, not general mutual exclusion.
3. volatile is suitable for simple state flags or publication patterns when the access pattern is appropriate.
4. Use synchronization or atomic/concurrent mechanisms when multiple-step invariants must be protected.

Java Code Example

```
class Worker {  
    private volatile boolean running = true;  
    void stop() { running = false; }  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

volatile is like a shared notice board everyone reliably sees; synchronized is like a room that only one worker can enter at a time.

Interview Tip: Never claim volatile makes arbitrary compound operations atomic.

Q52. What is deadlock?

Answer

1. Deadlock occurs when threads wait indefinitely for resources held by one another.
2. A common pattern is circular wait: thread A holds lock 1 and waits for lock 2 while thread B holds lock 2 and waits for lock 1.
3. Consistent lock ordering, minimizing lock scope, timeouts where appropriate, and higher-level concurrency utilities can reduce risk.
4. Deadlock differs from starvation and livelock even though all can prevent useful progress.

Java Code Example

```
synchronized (lockA) {  
    synchronized (lockB) {  
        // critical section  
    }  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Two people each holding one key and waiting forever for the other's key is a deadlock.

Interview Tip: Know the four Coffman conditions: mutual exclusion, hold-and-wait, no preemption, and circular wait.

Q53. Runnable vs Callable?

Answer

1. Runnable represents a task that does not return a result and cannot directly declare a checked exception from run().
2. Callable represents a task that returns a result and can throw checked exceptions from call().
3. Callable is commonly submitted to an ExecutorService and produces a Future.
4. Use Callable when the task's result or checked-failure contract matters.

Java Code Example

```
java.util.concurrent.ExecutorService ex = java.util.concurrent.Executors.newSingleThreadExecutor();  
java.util.concurrent.Future<Integer> f = ex.submit(() -> 42);  
ex.shutdown();
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Runnable is a job with no returned answer; Callable is a job that comes back with a result.

Interview Tip: Mention Future when explaining how Callable results are retrieved.

9. Wrapper Classes

Q54. What are wrapper classes?

Answer

1. Wrapper classes represent primitive values as objects, such as Integer for int and Double for double.
2. They are needed where APIs require objects, including generic collections such as List<Integer>.
3. Wrappers are immutable value objects.
4. Java supports autoboxing and unboxing to reduce the syntax needed when converting between primitives and wrappers.

Java Code Example

```
Integer x = 10; // autoboxing
int y = x; // unboxing
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A wrapper is a container that lets a primitive participate in object-oriented APIs.

Interview Tip: Remember the eight primitive-wrapper pairs, especially int/Integer and char/Character.

Q55. What is Integer caching?

Answer

1. Java may cache commonly used Integer objects and the Integer.valueOf API specifies caching for a range that includes -128 to 127.
2. Autoboxing uses wrapper conversion rules closely related to valueOf rather than always creating a new object.
3. Therefore == can appear true for small boxed integers but false for larger values or explicitly distinct objects.
4. equals() should be used for wrapper value comparison.

Java Code Example

```
Integer a = 100;
Integer b = 100;
Integer c = 1000;
Integer d = 1000;
System.out.println(a == b);
System.out.println(c == d);
Typical Output:
true
false
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A cache is like a reusable shelf for frequently requested small values.

Interview Tip: Do not generalize the Integer cache behavior to all wrapper values.

Q56. What happens when null is unboxed?

Answer

1. Unboxing converts a wrapper object to its primitive value.
2. If the wrapper reference is null, unboxing requires accessing a missing object value and throws NullPointerException.
3. This can happen implicitly in arithmetic, comparisons, or method calls expecting a primitive.
4. Null checks or suitable defaults are needed when nullable wrappers enter primitive operations.

Java Code Example

```
Integer x = null;  
// int y = x; // NullPointerException at runtime
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Trying to remove a value from an empty box causes a runtime failure.

Interview Tip: This is a frequent hidden NPE caused by compiler-generated unboxing.

10. Collections

Q57. What is the Java Collection Framework?

Answer

1. It is a set of interfaces, implementations, and algorithms for storing and manipulating groups of objects.
2. Core abstractions include Iterable, Collection, List, Set, Queue, Deque, and the separate Map hierarchy.
3. Implementations trade off ordering, duplication, lookup, insertion, sorting, concurrency, and memory behavior.
4. Choosing a collection should be based on required operations and guarantees rather than habit.

Java Code Example

```
java.util.List<String> names = new java.util.ArrayList<>();
names.add("Java");
names.add("Python");
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

It is a toolbox of standard data structures rather than one single collection type.

Interview Tip: Map is part of the Collections Framework conceptually, but Map does not extend Collection.

Q58. ArrayList vs LinkedList?

Answer

1. ArrayList stores elements in a resizable array and provides fast indexed access.
2. LinkedList is a doubly linked list and can be efficient for operations at known linked positions, but indexed access is linear.
3. ArrayList generally has better cache locality and is often the default List choice.
4. LinkedList's theoretical insertion advantages can disappear when locating the insertion position dominates the operation.

Java Code Example

```
java.util.List<Integer> list = new java.util.ArrayList<>();
list.add(10);
list.add(20);
System.out.println(list.get(1));
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

ArrayList is like numbered shelves; LinkedList is like train cars connected by links.

Interview Tip: Do not claim LinkedList insertion is always $O(1)$; finding the position can be $O(n)$.

Q59. HashMap vs TreeMap?

Answer

1. HashMap provides hash-based key lookup with expected average constant-time basic operations under good hashing.
2. TreeMap stores keys in sorted order using a balanced search-tree structure and typically offers $O(\log n)$ basic operations.
3. HashMap is appropriate when ordering is unnecessary and fast lookup is the priority.
4. TreeMap is appropriate when sorted keys or navigable range operations are required.

Java Code Example

```
java.util.Map<Integer,String> map = new java.util.TreeMap<>();
map.put(2, "B");
map.put(1, "A");
```

```
System.out.println(map);  
Expected Output:  
{1=A, 2=B}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

HashMap is like indexed bins chosen by a hash; TreeMap is like an ordered filing cabinet.

Interview Tip: State the ordering guarantee explicitly: HashMap does not promise sorted order.

Q60. How does HashMap work internally?

Answer

1. A HashMap uses a table of buckets and derives a hash-based location from the key.
2. A collision occurs when multiple keys map to the same bucket; the implementation resolves collisions using internal node structures.
3. Capacity and load factor influence when the table is resized, and resizing redistributes entries across a larger table.
4. Correct equals() and hashCode() implementations are essential for key lookup semantics.

Java Code Example

```
java.util.Map<String,Integer> m = new java.util.HashMap<>();  
m.put("A", 1);  
m.put("B", 2);  
System.out.println(m.get("A"));
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A key is hashed to find a candidate bucket, then equality checks identify the matching key.

Interview Tip: Avoid oversimplifying HashMap as 'just an array'; collision handling and resizing matter.

Q61. Why must equals() and hashCode() agree for HashMap keys?

Answer

1. HashMap first uses hash information to narrow the search to a bucket.
2. It then uses equality to identify the logically matching key among candidates.
3. If equal keys produce different hash codes, they can land in different buckets and fail to find one another.
4. Mutable key state is dangerous when it affects equals/hashCode after insertion because the entry can become difficult to retrieve.

Java Code Example

```
class Key {  
    final int id;  
    Key(int id) { this.id = id; }  
    public int hashCode() { return Integer.hashCode(id); }  
    public boolean equals(Object o) { return o instanceof Key k && id == k.id; }  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

The hash is the filing aisle; equals is the exact file check. The same logical key must reach the same aisle.

Interview Tip: This is one of the highest-frequency Java collection interview topics.

Q62. Comparable vs Comparator?

Answer

1. Comparable defines an object's natural ordering through compareTo().

2. Comparator defines an external ordering strategy through compare().
3. A class can have one natural ordering but many Comparator strategies.
4. Comparators are useful when sorting the same objects by different fields or business rules.

Java Code Example

```
java.util.List<String> names = new java.util.ArrayList<>();  
names.add("Bob"); names.add("Alice");  
names.sort(java.util.Comparator.comparingInt(String::length));
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Comparable is the object's default rule; Comparator is a separate sorting policy.

Interview Tip: Mention consistency with equals when designing orderings where appropriate.

11. Generics

Q63. What are generics?

Answer

1. Generics allow classes, interfaces, and methods to work with parameterized types.
2. They provide stronger compile-time type checking and reduce explicit casts in client code.
3. Generic type parameters are primarily a compile-time feature and are implemented through type erasure for ordinary Java generics.
4. Generics improve API clarity because the permitted element type is visible in the declaration.

Java Code Example

```
java.util.List<String> names = new java.util.ArrayList<>();
names.add("Java");
String s = names.get(0);
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Instead of a box labeled 'anything', you create a box labeled specifically for String values.

Interview Tip: Java generics are invariant by default; List is not a subtype of List.

Q64. What does ? extends mean?

Answer

1. ? extends T represents an unknown type that is T or a subtype of T.
2. It is useful when a structure is primarily a producer of T values.
3. You can safely read values as T, but you generally cannot add arbitrary T values because the exact subtype is unknown.
4. This is the 'Producer Extends' half of PECS.

Java Code Example

```
static double sum(java.util.List<? extends Number> nums) {
    double total = 0;
    for (Number n : nums) total += n.doubleValue();
    return total;
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A shelf may contain Number objects of one unknown specific subtype; you can take Numbers out safely.

Interview Tip: Explain why adding an arbitrary Number is unsafe.

Q65. What does ? super mean?

Answer

1. ? super T represents an unknown type that is T or a supertype of T.
2. It is useful when a structure is primarily a consumer of T values.
3. You can safely add T values, while reading generally gives Object because the exact supertype is unknown.
4. This is the 'Consumer Super' half of PECS.

Java Code Example

```
static void addScores(java.util.List<? super Integer> list) {
    list.add(10);
    list.add(20);
}
```

```
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A container accepting Integer could actually be a container of Number or Object, so adding Integer is safe.

Interview Tip: Remember PECS: Producer Extends, Consumer Super.

12. Java 8+ Features

Q66. What is a lambda expression?

Answer

1. A lambda is a concise way to represent behavior as a value for a functional interface.
2. Its syntax can omit parameter types and return syntax when the compiler can infer them.
3. Lambdas are commonly used with streams, collections, executors, and callback-style APIs.
4. A lambda does not create a new named class declaration; it supplies an implementation for a compatible functional interface target.

Java Code Example

```
java.util.function.Predicate<Integer> even = x -> x % 2 == 0;  
System.out.println(even.test(4));
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

It is like handing a method's behavior to another component without writing a full named class.

Interview Tip: A lambda requires a target functional interface type.

Q67. What is a functional interface?

Answer

1. A functional interface has exactly one abstract method.
2. It may still contain default and static methods.
3. Functional interfaces are the target types for lambda expressions and method references.
4. Common built-ins include Predicate, Consumer, Supplier, Function, BiFunction, UnaryOperator, and BinaryOperator.

Java Code Example

```
@FunctionalInterface  
interface Calculator { int add(int a, int b); }  
Calculator c = (a, b) -> a + b;
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

It is a one-operation contract that a lambda can implement.

Interview Tip: Object methods do not count as abstract methods for functional-interface compatibility.

Q68. What is the Stream API?

Answer

1. A Stream represents a pipeline for processing elements from a source such as a collection.
2. Intermediate operations such as map and filter build a pipeline and are generally lazy.
3. Terminal operations such as collect, forEach, count, or reduce trigger evaluation.
4. Streams can improve declarative data processing but are not automatically faster and should not be treated as collections.

Java Code Example

```
var result = java.util.List.of(1,2,3,4,5).stream()  
    .filter(x -> x % 2 == 0)  
    .map(x -> x * x)  
    .toList();  
System.out.println(result);  
Expected Output:
```

```
[4, 16]
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A stream is like a processing conveyor belt: values pass through operations rather than being a storage container.

Interview Tip: Distinguish stream pipeline from collection storage.

Q69. *map()* vs *filter()*?

Answer

1. map transforms each input element into an output element.
2. filter tests a predicate and keeps only elements that satisfy it.
3. map generally preserves the number of elements unless the mapping itself produces null or other values; filter can reduce the count.
4. They are often combined when selecting and transforming data.

Java Code Example

```
var result = java.util.List.of(1,2,3,4).stream()  
.filter(x -> x > 2)  
.map(x -> x * 10)  
.toList();  
System.out.println(result);  
Expected Output:  
[30, 40]
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

filter is a gate; map is a transformation machine.

Interview Tip: Do not describe map as 'selecting' elements.

Q70. *map()* vs *flatMap()*?

Answer

1. map transforms each element into one result value, which may itself be a collection or stream.
2. flatMap maps each element to a stream and then flattens the resulting streams into one stream.
3. flatMap is useful for nested structures such as List<List<String>>.
4. Using map where flattening is required leaves an extra nesting level.

Java Code Example

```
var nested = java.util.List.of(java.util.List.of("A","B"), java.util.List.of("C"));  
var flat = nested.stream().flatMap(java.util.Collection::stream).toList();  
System.out.println(flat);  
Expected Output:  
[A, B, C]
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

map creates boxes inside boxes; flatMap opens the inner boxes and combines their contents.

Interview Tip: Explain the shape of the input and output types.

Q71. *orElse()* vs *orElseGet()*?

Answer

1. orElse() receives a value expression that is evaluated before the method is invoked.
2. orElseGet() receives a Supplier and evaluates it only when the Optional is empty.

3. If the fallback computation is expensive or has side effects, `orElseGet` can avoid unnecessary work.
4. Both return a fallback value when the `Optional` is empty.

Java Code Example

```
String s = java.util.Optional.of("Java")
    .orElseGet(() -> "Fallback");
System.out.println(s);
Expected Output:
Java
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

`orElse` is like preparing the backup meal even when dinner is already available; `orElseGet` prepares it only if needed.

Interview Tip: The eager-vs-lazy fallback distinction is commonly tested.

13. JVM & Memory

Q72. What are the main JVM runtime data areas?

Answer

1. Each thread has its own PC register and JVM stack, while the heap is shared among threads.
2. The heap is the primary runtime area for objects and arrays.
3. Class metadata is stored in the method-area abstraction; in common HotSpot implementations since Java 8, class metadata is represented in native-memory Metaspace.
4. The JVM also has a native method stack and execution-engine components that operate outside the simple heap/stack picture.

Java Code Example

```
class Demo {  
    static int count = 1;  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        System.out.println(d);  
    }  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Think of each thread as having its own workbench while all threads share the application's object warehouse.

Interview Tip: Avoid the outdated statement that all class metadata is stored in the Java heap.

Q73. Where is an object stored?

Answer

1. An object is generally allocated in the JVM heap, although JVM optimizations may eliminate or scalar-replace allocations in some cases.
2. A reference variable may live in a stack frame when it is a local variable, or as part of an object when it is a field.
3. The Java language specification does not expose a simple guaranteed physical memory map for every optimization.
4. Interview answers should distinguish conceptual JVM areas from implementation-specific optimizations.

Java Code Example

```
class User { int id = 10; }  
User u = new User();
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Conceptually, u is a reference and the User object is on the heap.

Interview Tip: Say 'conceptually' or 'normally' rather than claiming every allocation must physically remain on the heap.

Q74. What happens when a method is called?

Answer

1. A stack frame is created for the executing method on the calling thread's JVM stack.
2. The frame contains local variables, an operand stack, and other implementation-defined information needed for execution.
3. Arguments and local references are associated with the frame, while referenced objects are generally in the heap.
4. When the method returns, its frame is popped and becomes unreachable as an execution frame.

Java Code Example

```
static int add(int a, int b) { return a + b; }  
public static void main(String[] args) {  
    System.out.println(add(2, 3));  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Calling a method is like opening a temporary workspace for that invocation; returning closes it.

Interview Tip: Do not say local objects themselves are 'stored on the stack' simply because their references are local.

Q75. What is JIT compilation?

Answer

1. JIT means Just-In-Time compilation and is part of the JVM execution strategy.
2. The JVM can identify frequently executed or 'hot' code and compile it to optimized native machine code.
3. This can improve long-running application performance compared with interpreting every bytecode instruction.
4. JIT optimizations can include inlining and other speculative/runtime optimizations, subject to deoptimization when assumptions change.

Java Code Example

```
for (int i = 0; i < 1_000_000; i++) {  
    Math.sqrt(i);  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A JVM can start with general execution and optimize heavily used paths after observing runtime behavior.

Interview Tip: Java performance is not simply 'bytecode interpreted'; modern JVMs use multiple execution techniques.

14. Garbage Collection

Q76. What is garbage collection?

Answer

1. Garbage collection automatically reclaims memory occupied by objects that are no longer reachable by the application.
2. The JVM determines eligibility using reachability rather than a simple reference-counting rule.
3. Modern collectors use different algorithms and may divide work across young and old object populations.
4. GC reduces manual memory-management errors but does not eliminate all memory leaks.

Java Code Example

```
Object obj = new Object();  
obj = null; // previous object may now be unreachable
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A lost object is like an abandoned item that no one can reach; the cleanup system can eventually reclaim its storage.

Interview Tip: 'Eligible for GC' does not mean 'immediately collected.'

Q77. Can Java have memory leaks?

Answer

1. Yes; GC cannot reclaim objects that are still reachable even if the application no longer logically needs them.
2. Common causes include unbounded caches, static collections, listeners that are never removed, and accidental long-lived references.
3. A memory leak can eventually contribute to OutOfMemoryError despite automatic garbage collection.
4. Profilers and heap analysis are used to identify unexpected retention paths.

Java Code Example

```
static final java.util.List<byte[]> cache = new java.util.ArrayList<>();  
// Continuously adding large arrays can retain memory.
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Keeping every old ticket in a permanent active-ticket list prevents cleanup even though the business no longer needs them.

Interview Tip: Java's GC manages reachability, not business-level usefulness.

15. Object Class

Q78. Why is Object important?

Answer

1. java.lang.Object is the root superclass for Java reference types unless a type ultimately derives through another class.
2. It defines fundamental methods such as equals, hashCode, toString, getClass, wait, notify, notifyAll, and clone.
3. Every class inherits Object behavior directly or indirectly.
4. Understanding Object is essential for collections, equality, debugging, synchronization, and reflection.

Java Code Example

```
class User { }  
User u = new User();  
System.out.println(u.toString());
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Object provides the baseline contract shared by ordinary Java classes.

Interview Tip: Know which Object methods are commonly overridden and why.

Q79. What is the equals/hashCode contract?

Answer

1. equals should be reflexive, symmetric, transitive, consistent, and false for null.
2. If a.equals(b) is true, a.hashCode() must equal b.hashCode().
3. Unequal objects are allowed to have the same hash code because collisions are legal.
4. Violating these rules can produce incorrect behavior in HashMap, HashSet, and related collections.

Java Code Example

```
System.out.println("A".equals("A"));  
System.out.println("A".hashCode() == "A".hashCode());
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Two logically equal objects must agree on their hash-based identity contract.

Interview Tip: Equal hash codes do not prove objects are equal.

16. Enums

Q80. What is an enum?

Answer

1. An enum defines a fixed set of named constants as a type.
2. Enum constants are instances of the enum type, not primitive integers.
3. Enums can have fields, constructors, methods, and implement interfaces.
4. They are preferable to scattered integer or string constants when the domain has a fixed set of values.

Java Code Example

```
enum Status { NEW, PAID, CANCELLED }  
Status s = Status.PAID;  
System.out.println(s);  
Expected Output:  
PAID
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

An enum is like a controlled menu containing only valid choices.

Interview Tip: Enum constructors are not called directly by application code.

17. Annotations

Q81. What are annotations?

Answer

1. Annotations attach metadata to program elements such as classes, methods, fields, and parameters.
2. The compiler, tools, frameworks, or runtime reflection can inspect annotations depending on retention.
3. Common built-ins include Override, Deprecated, and SuppressWarnings.
4. Custom annotations use @interface and can specify targets, retention, and elements.

Java Code Example

```
@Override
public String toString() {
    return "User";
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

An annotation is a label attached to code that tools or the runtime can interpret.

Interview Tip: Annotation presence alone does not automatically change behavior; some processor/framework must use it.

18. File I/O

Q82. Byte streams vs character streams?

Answer

1. InputStream and OutputStream work with bytes and are appropriate for binary data.
2. Reader and Writer work with character data and are appropriate for text.
3. Buffered wrappers reduce the cost of many small I/O operations by buffering data.
4. The correct choice depends on the data representation and encoding requirements.

Java Code Example

```
try (java.io.BufferedReader br = new java.io.BufferedReader(  
    new java.io.FileReader("data.txt"))) {  
    System.out.println(br.readLine());  
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A byte stream handles raw binary units; a character stream handles decoded text characters.

Interview Tip: For explicit charset control in modern code, prefer APIs such as Files with StandardCharsets where appropriate.

19. Date & Time

Q83. Why use java.time instead of the old Date/Calendar APIs?

Answer

1. java.time provides clearer types such as LocalDate, Instant, ZonedDateTime, Duration, and Period.
2. Most java.time value types are immutable and therefore easier to reason about in concurrent code.
3. The API separates concepts such as date-only values, machine instants, time zones, and amounts of time.
4. Formatting and parsing are handled through DateTimeFormatter rather than the older error-prone formatting APIs.

Java Code Example

```
java.time.LocalDate date = java.time.LocalDate.of(2026, 9, 1);
System.out.println(date);
Expected Output:
2026-09-01
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A calendar date, a clock time, and an instant on the global timeline are different concepts, so java.time gives them different types.

Interview Tip: Do not use LocalDateTime as a universal replacement for an Instant or ZonedDateTime.

20. JDBC

Q84. What is JDBC?

Answer

1. JDBC is Java's standard API for interacting with relational databases through drivers.
2. Typical flow involves obtaining a Connection, preparing/executing SQL, processing a ResultSet, and closing resources.
3. PreparedStatement is preferred for parameterized SQL because it separates SQL structure from parameter values.
4. Transactions use commit and rollback to control atomic groups of database operations.

Java Code Example

```
String sql = "SELECT id, name FROM users WHERE id = ?";
try (java.sql.Connection c = java.sql.DriverManager.getConnection(url, user, pass);
    java.sql.PreparedStatement ps = c.prepareStatement(sql)) {
    ps.setInt(1, 10);
    try (java.sql.ResultSet rs = ps.executeQuery()) {
        while (rs.next()) System.out.println(rs.getString("name"));
    }
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

JDBC is the standard bridge between Java application code and a relational database driver.

Interview Tip: Explain why PreparedStatement helps prevent SQL injection rather than claiming it 'encrypts SQL'.

Q85. executeQuery() vs executeUpdate()?

Answer

1. executeQuery() is normally used for statements that return a ResultSet, such as SELECT.
2. executeUpdate() is normally used for INSERT, UPDATE, DELETE, and some DDL operations and returns an update count where applicable.
3. The exact JDBC method should match the SQL operation and driver's supported behavior.
4. Both should be executed through properly parameterized statements when user input is involved.

Java Code Example

```
PreparedStatement ps = connection.prepareStatement("UPDATE users SET name=? WHERE id=?");
ps.setString(1, "A");
ps.setInt(2, 1);
int count = ps.executeUpdate();
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A query reads a result set; an update operation reports how many rows were affected.

Interview Tip: Know the return type and common SQL mapping of both methods.

21. Modern Java

Q86. What is a record?

Answer

1. A record is a concise Java type for modeling data-centric objects with a fixed set of components.
2. The compiler provides accessors, equals, hashCode, and toString implementations based on record components.
3. Records are shallowly immutable in the sense that their component references cannot be reassigned after construction, but referenced mutable objects can still change.
4. Records are useful for DTO-like data carriers but are not a replacement for every class.

Java Code Example

```
record User(int id, String name) {}  
User u = new User(1, "Anshu");  
System.out.println(u.name());  
Expected Output:  
Anshu
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A record is like a compact data contract where the common boilerplate is generated for you.

Interview Tip: Record components are accessed with component methods such as name(), not JavaBean getName().

Q87. What is var?

Answer

1. var allows local variable type inference when the compiler can determine the variable's static type from its initializer.
2. It does not make Java dynamically typed; the variable still has a compile-time type.
3. var cannot be used for fields, method parameters, or return types.
4. Use it when the inferred type remains clear; avoid it when it harms readability.

Java Code Example

```
var name = "Java";  
var count = 10;  
System.out.println(name + " " + count);
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

The compiler knows the types even though the source omits the repeated type name.

Interview Tip: Say 'local variable type inference,' not 'dynamic typing.'

Q88. What are sealed classes?

Answer

1. A sealed class or interface restricts which types may directly extend or implement it.
2. Permitted subclasses are declared using permits or can be inferred in some cases based on the declaration context.
3. Sealing can make a domain hierarchy explicit and improve reasoning about exhaustive handling.
4. A permitted subtype must satisfy the required final, sealed, or non-sealed modifier rules.

Java Code Example

```
sealed interface Shape permits Circle, Square {}  
final class Circle implements Shape {}  
final class Square implements Shape {}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

It is like a company directory where only explicitly authorized departments can belong to a hierarchy.

Interview Tip: Know final, sealed, and non-sealed as the three relevant subclassing outcomes.

22. Tricky Interview Questions

Q89. What is the output of String pool comparison?

Answer

1. String literals with identical contents can refer to the same pooled String instance.
2. Therefore two identical literals commonly compare true with ==.
3. A String created with new is a distinct object even if its contents match a pooled literal.
4. equals() compares contents and should be used for logical string equality.

Java Code Example

```
String a = "Java";
String b = "Java";
System.out.println(a == b);
Expected Output:
true
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Two identical labels from the shared pool can point to one object.

Interview Tip: Explain the pool rather than memorizing the output.

Q90. What is the output of wrapper comparison?

Answer

1. Autoboxing can reuse cached wrapper instances for certain values.
2. Integer caching includes the range -128 through 127 by specification for valueOf-related behavior.
3. Larger boxed values are not guaranteed to share the same object.
4. Use equals() for wrapper value comparison.

Java Code Example

```
Integer a = 100;
Integer b = 100;
Integer c = 1000;
Integer d = 1000;
System.out.println(a == b);
System.out.println(c == d);
Typical Output:
true
false
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Small cached objects can be shared while larger values can produce distinct objects.

Interview Tip: Never build application logic around wrapper ==.

Q91. Can a static method be overridden?

Answer

1. No, static methods are hidden rather than overridden.
2. Static method selection is based on the reference/class context at compile time rather than runtime dynamic dispatch.
3. A child can declare a static method with the same signature, but it does not participate in runtime polymorphism like an instance method.
4. Mixing static hiding and instance overriding in interview examples is a common trap.

Java Code Example

```
class A { static void show() { System.out.println("A"); } }  
class B extends A { static void show() { System.out.println("B"); } }  
A x = new B();  
x.show();  
Expected Output:  
A
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Static calls are associated with the reference/class context rather than the runtime object.

Interview Tip: Say 'method hiding,' not 'static method overriding.'

Q92. Can an abstract class have a constructor?

Answer

1. Yes, an abstract class can declare constructors.
2. The constructor runs as part of constructing a concrete subclass object.
3. You cannot directly instantiate the abstract class itself.
4. Constructors are useful for initializing common state shared by subclasses.

Java Code Example

```
abstract class A {  
A() { System.out.println("A constructor"); }  
}  
class B extends A { B() { System.out.println("B constructor"); } }  
Expected Output when new B() is executed:  
A constructor  
B constructor
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A foundation can be initialized even though the foundation type itself cannot be instantiated independently.

Interview Tip: Abstract means incomplete for direct instantiation, not 'constructorless.'

Q93. What happens with parent reference and child object?

Answer

1. A parent reference can point to a child object through upcasting.
2. Overridden instance methods are dynamically dispatched to the child implementation.
3. Child-specific members are not directly accessible through the parent reference without a safe downcast.
4. Downcasting should be used only when the runtime object is actually compatible with the target type.

Java Code Example

```
class Parent { void show() { System.out.println("Parent"); } }  
class Child extends Parent { void show() { System.out.println("Child"); } void extra() {} }  
Parent p = new Child();  
p.show();  
Expected Output:  
Child
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

The reference controls what can be named; the runtime object controls overridden instance behavior.

Interview Tip: This distinction is central to runtime polymorphism.

Q94. Why can HashSet contain only one of two equal objects?

Answer

1. HashSet uses hashCode to locate candidate buckets and equals to determine logical duplication.
2. If two objects are logically equal and correctly implement the contract, the second is treated as a duplicate.
3. If equals/hashCode are inconsistent, collection behavior can become surprising or incorrect.
4. Mutable fields used in hashing should generally not change while the object is stored as a key/set element.

Java Code Example

```
java.util.Set<String> s = new java.util.HashSet<>();
s.add(new String("Java"));
s.add(new String("Java"));
System.out.println(s.size());
Expected Output:
1
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

The two distinct String objects have equal content and compatible hash/equality behavior.

Interview Tip: Distinct objects do not imply distinct Set elements.

Q95. What happens if catch blocks are in the wrong order?

Answer

1. A catch block for a broader exception type can make a later catch for its subclass unreachable.
2. The compiler detects unreachable catch clauses in these common cases.
3. Specific exceptions should generally be caught before their broader parent types.
4. This ordering follows the exception inheritance hierarchy.

Java Code Example

```
try {
    throw new java.io.IOException();
} catch (Exception e) {
    System.out.println("Exception");
} // a later IOException catch would be unreachable
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

The broader catch already handles the subclass, so the later specific handler can never be reached.

Interview Tip: Order catches from most specific to most general.

Q96. What is the difference between final, finally, and finalize?

Answer

1. final is a keyword for restricting reassignment, overriding, or inheritance depending on where it is used.
2. finally is an exception-handling block intended for cleanup code that normally runs after try/catch control flow.
3. finalize was a historical Object method associated with GC-related cleanup and has been deprecated for removal; modern code should not rely on it.
4. The three names are related only syntactically, not conceptually.

Java Code Example

```
final int x = 10;
try { System.out.println(x); }
finally { System.out.println("cleanup"); }
Expected Output:
10
cleanup
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

They sound alike but belong to different language/runtime mechanisms.

Interview Tip: Modern Java interview answers should explicitly treat finalization as obsolete/deprecated.

23. Scenario-Based Questions

Q97. How would you design a banking system using OOP?

Answer

1. Create focused domain classes such as Account, Customer, Transaction, and services rather than one giant class.
2. Encapsulate balances and enforce invariants through controlled operations such as deposit and withdraw.
3. Use interfaces for replaceable policies such as payment or notification mechanisms and composition for collaboration.
4. Separate persistence, business logic, and presentation concerns so the model remains testable and maintainable.

Java Code Example

```
interface AccountService { void transfer(long from, long to, double amount); }
class BankService implements AccountService {
    public void transfer(long from, long to, double amount) { /* validate + transact */ }
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A bank needs controlled account state, explicit transactions, and replaceable infrastructure components.

Interview Tip: Discuss thread safety and transaction boundaries if the interviewer moves from OOP into system behavior.

Q98. Which collection would you use for fast lookup?

Answer

1. For key-based lookup, HashMap is usually the first choice when ordering is unnecessary.
2. HashSet is suitable when the requirement is fast membership checking without key-value pairs.
3. TreeMap or TreeSet is appropriate when sorted order or navigable range operations are required.
4. ConcurrentHashMap is appropriate when shared concurrent access requires a concurrent map rather than external global locking.

Java Code Example

```
java.util.Map<String, Integer> counts = new java.util.HashMap<>();
counts.put("Java", 5);
System.out.println(counts.get("Java"));
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Choose the data structure from the operation and guarantees, not from a generic claim that one collection is always fastest.

Interview Tip: State expected complexity and concurrency/order requirements.

Q99. How would you process millions of records?

Answer

1. Avoid loading all records into memory if the workload can be streamed or batched.
2. Use pagination, streaming I/O, database-side filtering, and batch processing where appropriate.
3. Choose data structures with memory and lookup costs appropriate to the workload.
4. Measure throughput, memory, GC behavior, and database/network bottlenecks rather than optimizing Java code blindly.

Java Code Example

```
try (java.io.BufferedReader br =
    java.nio.file.Files.newBufferedReader(java.nio.file.Path.of("data.csv"))) {
```

```
String line;
while ((line = br.readLine()) != null) {
    // process one record
}
}
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A conveyor belt processes records continuously instead of putting millions of boxes into one room.

Interview Tip: Mention backpressure or bounded batching when downstream systems can be slower than input.

Q100. How would you prevent duplicate data in an application?

Answer

1. Choose a data structure such as Set when uniqueness is a local in-memory requirement.
2. For database data, enforce uniqueness at the database level with appropriate unique constraints or indexes.
3. Validate at the application layer for user feedback, but do not rely only on a race-prone pre-check for correctness.
4. In distributed systems, idempotency keys or transactional constraints may be needed to prevent duplicate operations.

Java Code Example

```
java.util.Set<String> ids = new java.util.HashSet<>();
ids.add("A");
ids.add("A");
System.out.println(ids.size());
Expected Output:
1
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

A Set models uniqueness locally; a database constraint provides authoritative persistence-level uniqueness.

Interview Tip: Application validation and database enforcement solve different layers of the problem.

Q101. How would you handle concurrent access to shared state?

Answer

1. First identify whether the state truly needs to be shared and whether immutable design can remove the race.
2. Use synchronized blocks, locks, atomic classes, or concurrent collections according to the access pattern.
3. Minimize critical-section size and avoid holding locks while performing slow external operations.
4. Test concurrency behavior under realistic contention because timing bugs often do not reproduce in simple tests.

Java Code Example

```
java.util.concurrent.atomic.AtomicInteger count = new java.util.concurrent.atomic.AtomicInteger();
count.incrementAndGet();
System.out.println(count.get());
```

This demonstrates the core concept in a small interview-friendly example.

Real-Life Example

Instead of letting many workers edit one number unsafely, an atomic abstraction provides a controlled update operation.

Interview Tip: Choose the smallest synchronization mechanism that correctly protects the invariant.

Quick Comparison Tables

JDK vs JRE vs JVM

Concept	Purpose
JVM	Executes Java bytecode and manages runtime execution.
JRE	Conceptual runtime environment: JVM plus runtime libraries.
JDK	Development kit: compiler/tools plus runtime components.

String vs StringBuilder vs StringBuffer

Type	Mutable?	Typical use
String	No	Immutable text/value.
StringBuilder	Yes	Efficient mutable text building.
StringBuffer	Yes	Synchronized mutable text API when required.

ArrayList vs LinkedList

Aspect	ArrayList	LinkedList
Structure	Resizable array	Doubly linked list
Indexed access	Fast, O(1) typical	O(n)
Typical default	Often preferred	Use when linked-list semantics genuinely help

HashMap vs TreeMap

Aspect	HashMap	TreeMap
Ordering	No sorted-order guarantee	Sorted by key
Typical basic lookup	Expected O(1)	O(log n)
Best when	Fast key lookup	Sorted/navigable keys

Overloading vs Overriding

Aspect	Overloading	Overriding
Binding	Compile time	Runtime for instance methods
Where	Usually same class/API	Subclass implementation
Return type alone	Not enough	Must have compatible return type

throw vs throws

Concept	Meaning
throw	Actually throws a Throwable object.
throws	Declares exceptions in a method/constructor signature.

sleep vs wait

Concept	Meaning
sleep()	Pauses current thread; does not release monitors.
wait()	Releases the relevant monitor while waiting; used for coordination.

Java Interview Revision Checklist

- Data types, ranges, defaults, casting, widening and narrowing
- Variables, scope, lifetime, static vs instance
- Operators, precedence, control statements and loops
- Java history, platform independence, bytecode, JDK/JRE/JVM
- Strings, String pool, immutability, == vs equals
- Methods, overloading, overriding, pass-by-value
- Constructors, this, super, final, static
- OOP: encapsulation, inheritance, polymorphism, abstraction
- Interfaces, abstract classes, composition, IS-A/HAS-A
- equals/hashCode, Object class, toString
- Access modifiers and overriding restrictions
- Exception hierarchy, checked/unchecked, throw/throws
- try-with-resources, AutoCloseable, suppressed exceptions
- Threads, lifecycle, start/run, synchronization
- wait/notify, sleep, volatile, race conditions, deadlock
- ExecutorService, Callable, Future, concurrent collections
- Wrapper classes, autoboxing/unboxing, Integer cache
- List/Set/Map/Queue and implementation trade-offs
- HashMap internals, hashing, collisions, capacity, load factor
- Comparable vs Comparator
- Generics, wildcards, bounds and PECS
- Lambdas and functional interfaces
- Stream API, lazy evaluation, map/filter/flatMap/reduce
- Optional and orElse/orElseGet
- JVM memory areas, class loading, JIT
- Garbage collection, reachability, memory leaks
- Enums and annotations
- File I/O, byte vs character streams, serialization
- java.time API
- JDBC, PreparedStatement, ResultSet, transactions
- Records, sealed classes, var, switch expressions
- Output-based and memory questions
- Real-world OOP and concurrency scenarios

Final Interview Rule

Define the concept first, explain why it exists, show a short example, then discuss trade-offs and a real-world use case. For output questions, trace compile-time type, runtime object, evaluation order, and relevant JVM/collection rules before stating the output.